

Chatbot modul

Fejlesztési terv és specifikáció

Architektúra

A modul öt fő rétegre bontható. A rétegek célja, hogy a DNN-be ágyazott frontend UI, a fejlesztői teszt Kliens, az LLM-szolgáltató és az adatbázis-lekérdezések külön fejleszthetők maradjanak.

- DNN frontend: chatbot UI, amely a backend API-t hívja.
- Fejlesztői teszt Kliens: statikus `index.html`, lokális kézi teszteléshez.
- API réteg: `src/main.py`, FastAPI app request és response modellekkel.
- LLM orchestration: `src/generate.py`, `src/llm/router.py`, `src/llm/formatter.py`.
- Data access: `src/data/sql_tools.py`, `src/database/config.py`, `src/database/engine.py`.

Rétegek feladatai

Réteg	Felelősség	Fő fájlok / rendszer
DNN CMS chatbot UI	Frontend felhasználói interakció és a chatbot beágyazása a webáruház CMS környezetébe.	DNN CMS komponens
Lokális teszt Kliens	Kérdés beküldése, válasz és tool metaadat ellenőrzése fejlesztés közben.	<code>index.html</code>
API	Endpointok, Pydantic validáció, hibák HTTP response-szá alakítása.	<code>src/main.py</code>
LLM orchestration	Message összeállítás, router modell futtatása, tool callok feloldása, végső answer előállítás.	<code>src/generate.py</code> , <code>src/llm/*</code>
SQL integráció	SQL Server kapcsolat, termékadatok lekérése, JSON-kompatibilis adatkonverzió.	<code>src/data/sql_tools.py</code> , <code>src/database/*</code>
Konfiguráció	LLM és SQL Server paraméterek betöltése környezeti változókból. A Langfuse oldalon nincs saját konfigurációs réteg, csak a wrapper importált.	<code>.env</code> , <code>.env.sample</code>

User Journey

Production környezetben a felhasználó a DNN CMS-be ágyazott chatbot felületen írja be a kérdését. A DNN komponens JSON requestet küld a backend `POST /question` endpointjára. A request tartalmazza az aktuális kérdést, opcionálisan a beszélgetési előzményeket, illetve a `history_window` értéket.

A router modell eldönti, hogy elég-e közvetlen LLM answer, vagy szükség van adatbázisos tool callra. Tipikus webshopos kérdés lehet egy konkrét modellautó ára, készlete, részletes adatai, hasonló alternatívái, akciós termékek listázása, kategória szerinti keresés vagy több modell összehasonlítása.

A backend JSON response-t ad vissza. A DNN UI ezt a CMS design rendszeréhez igazítva jeleníti meg, míg az `index.html` tesztkliens egyszerű szöveggént és listaként rajzolja ki.

Felhasználói esetek

Eset	Leírás	Elvárt eredmény
Általános webshop kérdés	A felhasználó a modellautó-webáruházhoz kapcsolódó általános kérdést tesz fel.	A router nem hív toolt, a modell közvetlen answer-t ad.
Népszerű termékek	A felhasználó top, népszerű vagy leggyakrabban foglalt termékekre kérdez.	A <code>get_hot_products</code> tool lefut, az eredményből a formatter készít választ.
Termékinformáció	A felhasználó egy adott modellautó árát, készletét, részleteit vagy hasonló termékeket kérdez.	A tervezett product toolok egyike ad strukturált adatot a formatternek.
Beszélgetési folytatás	A felhasználó az előző answerre utal vissza.	A kliens átadja a legutóbbi előzményeket, a backend a beállított window méretéig felhasználja őket.
Kézi fejlesztői teszt	A fejlesztő az <code>index.html</code> fájlból teszteli az API-t.	A tesztkliens megjeleníti az API response-t, de nem production UI.

Adatbázis

A modul a termékadatokat meglévő SQL Server adatbázisból olvassa. A connection stringet a `DB_CONNECTION_STRING` környezeti változó tartalmazza ODBC formátumban. A tervezett product toolok ugyanarra a data access rétegre épülnek, de több lekérdezést, szűrést és összehasonlító műveletet vezetnek be.

Kapcsolat

Elem	Leírás
<code>DB_CONNECTION_STRING</code>	ODBC connection string a projekt gyökerben lévő <code>.env</code> fájlban.
<code>get_database_url()</code>	A connection stringet URL-kódolja, és <code>mssql+pyodbc:///?odbc_connect=...</code> formára alakítja.
<code>get_engine()</code>	Singleton SQLAlchemy engine-t ad vissza <code>pool_pre_ping=True</code> beállítással.
<code>SessionLocal</code>	SQLAlchemy session factory későbbi tranzakciós vagy repository-alapú fejlesztésekhez.

Várható termékmezők

Mező	Típus	Leírás
ProductId	int/string	Termékazonosító toolok közötti hivatkozáshoz és összehasonlításhoz.
ProductName	string	A modellautó neve.
Category	string	Termékkategória, például versenyautó, klasszikus autó vagy kiegészítő.
Scale	string	Méretarány, például 1:18, 1:24 vagy 1:43.
Brand	string	Gyártó vagy márka.
Price	decimal	Aktuális ár, alapértelmezett pénznemként Ft kezeléssel.
Stock	int/string	Készletmennyiség vagy elérhetőségi státusz.
Description	string	Rövid vagy részletes termékleírás.

Backend szolgáltatások

A backend jelenleg függvényalapú modulokból áll. A fő logikai egységek: LLM client, router, formatter, tool resolver és SQL data access.

Chat orchestration - generate_response

A `generate_response(question, history, history_window)` a chatbot központi belépési pontja. Előkészíti az LLM klienst, ellenőrzi a modellnevet, összeállítja a message listát, futtatja a routert, feloldja az esetleges tool callt, majd visszaadja a végső answer-t.

Paraméter	Típus	Leírás
question	string	Az aktuális felhasználói kérdés.
history	lista vagy null	Korábbi user és assistant üzenetek. Csak érvényes role és nem üres content kerül továbbításra.
history_window	int	Legfeljebb hány friss előzmény kerüljön be a modell kontextusába. API-szinten 0 és 20 közötti érték.

LLM client és Langfuse wrapper

Az LLM kliens a `src/llm/client.py` fájlban jön létre. A kód a szokásos OpenAI-kompatibilis kliens helyett a `langfuse.openai` wrapperből importált `OpenAI` osztályt használja. Emiatt az alkalmazás oldali API ugyanaz marad, mint egy OpenAI-kompatibilis kliensnél: a router és a formatter továbbra is `client.chat.completions.create(...)` hívásokkal dolgozik.

A Langfuse szerepe ebben a projektben jelenleg az alap LLM observability támogatása. Nekünk ez azért hasznos, mert a chatbot futásai két modellhívásból is állhatnak: először a router dönt a tool használatáról, majd a formatter megfogalmazza a végső választ. A wrapper segítségével ezek a hívások később visszakereshetők és debugolhatók lehetnek: láthatóvá válhat, milyen input ment a modellnek, milyen answer érkezett vissza, melyik modell futott, illetve hol volt lassú vagy hibás a folyamat.

Fontos korlát, hogy a projektben nincs külön saját Langfuse konfigurációs réteg vagy egyedi Langfuse logika. Nem implementáltunk custom trace-eket, tageket, score-okat, prompt managementet, evaluation flow-t vagy dashboardot. A jelenlegi megoldás mindössze annyi, hogy az OpenAI API wrapper Langfuse-os, így az alap megfigyelhetőség a wrapper képességeire épül.

Router modell

A router a `ROUTER_SYSTEM_PROMPT` alapján a modellautó-webáruház kontextusában dönt. A router és

formatter system promptjai a `src/prompts/system.py` fájlban vannak, nem a `.env`-ből érkeznek. A router feladata, hogy a felhasználói kérdéshez a legszűkebb szükséges toolt válassza ki, vagy tool nélkül adjon direkt answer-t.

Jelenlegi és tervezett toolok

Tool	Állapot	Feladat
<code>get_hot_products</code>	Meglévő alap tool	Népszerű, top vagy legtöbbször foglalt termékek lekérése.
<code>get_recommendation</code>	Előkészített bővítési pont	Termékajánlás későbbi szabályok és termékadatok alapján.
Product toolok	Tervezett	Termékkeresés, ár, készlet, akciók, hasonló termékek és összehasonlítás kezelése.

Tervezett product toolok

A következő toolok a modellautó-webshop asszisztensi működését bővítik. A cél az, hogy a chatbot termékadatokra ne szabadon generált szövegből válaszoljon, hanem ellenőrzött, strukturált adatot kapjon a backendtől, majd azt a formatter alakítsa természetes válasszá.

Tool	Cél	Tervezett input	Tervezett output
<code>get_product_by_name</code>	Konkrét modellautó lekérése pontos vagy közel pontos terméknév alapján.	<code>name</code>	Egy termék alapadatai: azonosító, név, kategória, ár, készlet, rövid leírás.
<code>search_products</code>	Szabad szavas keresés márka, típus, méretarány vagy kulcsszó alapján.	<code>query</code> , <code>limit</code>	Találati lista relevancia szerinti sorrendben.
<code>get_products_by_category</code>	Terméklista lekérése kategória szerint, például versenyautó, klasszikus autó vagy kiegészítő.	<code>category</code> , <code>limit</code>	A kategóriába tartozó termékek rövidített adatai.
<code>get_product_stock</code>	Készletinformáció lekérése egy adott modellautóhoz.	<code>product_id</code> vagy <code>name</code>	Aktuális készlet, elérhetőségi státusz, opcionálisan várható utánpótlás.
<code>get_product_price</code>	Aktuális ár lekérése egy adott termékhez.	<code>product_id</code> vagy <code>name</code>	Listaár, akciós ár, pénznem és árvényesség.
<code>get_discounted_products</code>	Akciós vagy kedvezményes modellautók listázása.	<code>limit</code> , <code>category</code>	Akciós terméklista eredeti és kedvezményes árakkal.
<code>get_products_by_price_range</code>	Termékek szűrése árintervallum alapján.	<code>min_price</code> , <code>max_price</code> , <code>category</code>	A megadott árhatárok közé eső termékek.

<code>get_product_details</code>	Részletes termékadatlap lekérése.	<code>product_id</code> vagy <code>name</code>	Leírás, méretarány, gyártó, anyag, elérhetőség, ár és további metaadatok.
<code>get_similar_products</code>	Hasonló modellautók ajánlása egy kiválasztott termék alapján.	<code>product_id</code> , <code>limit</code>	Hasonló kategóriájú, árú, márkájú vagy méretarányú termékek.
<code>compare_products</code>	Több modellautó összehasonlítása vásárlási döntéshez.	<code>product_ids</code> vagy <code>names</code>	Összehasonlító adatok ár, készlet, kategória, méretarány és fő jellemzők szerint.

Formatter modell

A `run_formatter_model()` egy második LLM-hívással készíti el a végső, felhasználónak szánt `answer`-t. Bemenete egy JSON objektum: a `user question`, a `route` neve, az `SQL` vagy `tool result`, illetve a `direkt answer`. A `formatter` prompt előírja, hogy a válasz a kapott adatokra támaszkodjon, ne találjon ki termékadatokat, és markdown nélkül, természetes nyelven fogalmazzon.

REST API

A backend FastAPI appként fut. A CORS beállítás jelenleg fejlesztéshez teljesen nyitott: minden `origin`, `method` és `header` engedélyezett.

GET /

Visszaadja a projekt gyökerében található `index.html` fejlesztői tesztklient. Ez nem `production` felület, csak a backend gyors kézi ellenőrzésére szolgál.

GET /health

Egyszerű `health check` endpoint. Sikeres működés esetén a `response`: `{ "status": "ok" }`.

POST /question

A chatbot fő endpointja. JSON requestet fogad, lefuttatja a válaszgenerálást, majd JSON `response`-t ad vissza.

Request field	Típus	Validáció	Leírás
<code>question</code>	string	1-4000 karakter	Az aktuális kérdés.
<code>history</code>	lista	Opcionális, alapérték üres lista	Korábbi üzenetek <code>user</code> vagy <code>assistant</code> role-lal.
<code>history_window</code>	int	0-20, alapérték 4	A figyelembe vett legutóbbi üzenetek száma.

Response field	Típus	Leírás
<code>answer</code>	string	A felhasználónak megjelenítendő végső válasz.
<code>tool_used</code>	string vagy null	A használt <code>tool</code> neve, ha történt <code>tool call</code> .
<code>data</code>	object vagy null	Strukturált kiegészítő adat, például terméklista, ár, készlet vagy összehasonlítás.

Tool response-ok tervezett szerkezete

A product toolok egységes, JSON-kompatibilis response-t adnak. A formatter minden esetben ebből az objektumból dolgozik. A production DNN UI dönthet úgy, hogy a `data` mezőből külön termékkártyákat vagy összehasonlító nézetet rajzol.

Szerkezet	Használat
<pre>{ "product": { ... } }</pre>	Egy konkrét termék részletei, ára vagy készlete.
<pre>{ "products": [...] }</pre>	Keresés, kategória, akciós termékek, árintervallum vagy hasonló termékek listája.
<pre>{ "comparison": [...] }</pre>	Több termék összehasonlításának strukturált eredménye.

UI-k

DNN CMS-be ágyazott chatbot

A frontend UI a React alapon a DNN rendszer részeként jelenik meg, ezért a végleges layout, jogosultsági kontextus és stílus a CMS rendszerhez igazodik. A backend szempontjából a DNN komponens feladata a JSON request összeállítása, az API hívása, majd a response megjelenítése.

- A DNN komponens hívja a `POST /question` endpointot.
- A beszélgetési előzményeket a DNN oldali klienslogika adhatja át.
- A toolból érkező strukturált adatokat a CMS design rendszerének megfelelően kell kirajzolni.

index.html fejlesztői tesztkliens

A repository gyökerében található `index.html` önálló, statikus HTML fájl. Célja kizárólag az, hogy fejlesztés közben gyorsan ellenőrizhető legyen az API működése DNN környezet nélkül.

Prompt terv

A rendszer két külön, kódban tárolt system promptot használ. Ezek nem a `.env` konfiguráció részei, hanem a `src/prompts/system.py` fájlban találhatók.

Router prompt

- A chatbot maradjon a modellautó-webáruház kontextusában.
- Csak akkor hívjon toolt, ha a kérdés egyértelműen illeszkedik egy elérhető vagy tervezett toolhoz.
- Konkrét termékadatoknál részesítse előnyben a strukturált adatot adó toolokat.
- Általános, információs vagy nem támogatott kérdésnél adjon direkt answer-t.

Formatter prompt

- A kapott tool result alapján válaszoljon, ne találjon ki terméknevet, árat, készletet vagy kedvezményt.
- Az árakat forintként értelmezze, ha ármező érkezik.
- Legyen barátságos, természetes és vásárlási döntést segítő.
- Ne használjon markdown formázást, mert a jelenlegi tesztkliens sima szöveggént jeleníti meg a választ.

Konfiguráció

A projekt a gyökérben lévő `.env` fájlból olvassa a futásidejű beállításokat. A titkos értékeket nem szabad repositoryba commitolni. A `.env.sample` csak kulcsneveket és mintaértékeket tartalmaz.

Változó	Kötelező?	Leírás
API_KEY	Igen	Az OpenAI-kompatibilis LLM szolgáltató API kulcsa.
BASE_URL	Igen	Az LLM API base URL-je, például OpenRouter vagy saját gateway.
LLM_MODEL	Igen	A használni kívánt modell azonosítója.
LANGFUSE_SECRET_KEY	Wrapper oldali	A projektkód nem olvassa közvetlenül. Ha a futtatási környezetben be van állítva, a Langfuse wrapper használhatja.
LANGFUSE_PUBLIC_KEY	Wrapper oldali	A projektkód nem olvassa közvetlenül. A Langfuse wrapper környezeti változóként kezelheti.
LANGFUSE_BASE_URL	Wrapper oldali	A projektben nincs külön Langfuse config modul; az érték csak a wrapper környezetéhez tartozik.
DB_CONNECTION_STRING	Igen SQL toolhoz	SQL Server ODBC connection string.

Biztonság és adatvédelem

A chatbot külső LLM szolgáltatóhoz továbbítja a felhasználói kérdéseket és a beszélgetési előzmények egy részét, ezért production bevezetésnél adatvédelmi és jogosultsági szabályokat kell alkalmazni.

- A `.env` fájlban lévő API kulcsok és adatbázis connection string titkos adatok.
- A teljesen nyitott CORS beállítást DNN domainre kell korlátozni.
- A backend előtt jelenleg nincs saját auth réteg; productionben ezt a DNN környezettel össze kell hangolni.
- A felhasználói kérdések az LLM szolgáltatóhoz kerülnek, ezért személyes adatot csak szabályozott környezetben szabad beküldeni.
- A tervezett product tooloknál minden SQL-lekérdezést paraméterezetten kell implementálni.

Futtatás és deployment

Lokális fejlesztéshez a projekt az alábbi lépésekkel futtatható.

- Függőségek telepítése: `uv sync`.
- `.env` létrehozása a `.env.sample` alapján.
- API indítása: `uv run uvicorn main:app --app-dir src --reload --port 8080`.
- Lokális tesztkliens: `http://localhost:8080`.
- Health check: `http://localhost:8080/health`.
- Chat endpoint: `POST http://localhost:8080/question`.

Production használatban a DNN CMS-be ágyazott frontend hívja ugyanezt az API-t. A `deploy.ps1` script friss kódot húz, szinkronizálja a függőségeket, majd újraindítja a `closedai-chatbot-api` Windows service-t.

Tesztelési terv

A repositoryban jelenleg nem található külön tesztcsomag, ezért célzott unit és integration teszteket kell hozzáadni. A teszteknel az LLM-et és az adatbázist érdemes mockolni, hogy a futás ne függjön külső szolgáltatástól.

Tesztterület	Javaolt ellenőrzések
Message build	<code>_build_messages</code> : history window, üres vagy hibás history elemek kihagyása.

Router resolver	Direkt answer, meglévő tool, tervezett tool, ismeretlen tool és hibás argumentum kezelése.
Product toolok	Névkeresés, kategóriaszűrés, árintervallum, készlet, akciók, hasonló termékek és összehasonlítás tesztadatai.
SQL tool	Paraméterezett lekérdezések, Decimal és date/datetime JSON-kompatibilis konverzió, üres result kezelése.
API	/health response, /question request validáció, 500-as hibakezelés mockolt kivételekkel.
DNN production integráció	A DNN komponens által küldött request szerkezete, auth kontextus, CORS és felhasználói hibaüzenetek.

Kockázatok és korlátok

- Az LLM válaszai szolgáltatótól, modelltől és prompttól függenek, ezért nem teljesen determinisztikusak.
- A külső LLM szolgáltató kiesése vagy lassulása közvetlenül érinti a chatbot válaszidejét.
- A tervezett product toolokhoz pontos adatmodellre és egységes response szerkezetre lesz szükség.
- Nincs beépített rate limit, backend oldali user azonosítás vagy audit log.
- Az API szinkron endpointként van megírva, miközben LLM- és adatbázishívásokat végez. Nagyobb terhelésnél async vagy háttérfolyamatos kialakítás javasolt.

Fájlstruktúra

Fájl	Szerep
src/main.py	FastAPI app, request/response modellek, HTTP endpointok.
src/generate.py	A chatbot válaszgenerálásának fő folyamata.
src/llm/client.py	LLM client létrehozása Langfuse OpenAI wrapperrel és környezeti változókból.
src/llm/router.py	Tool deklarációk és router modell hívása.
src/llm/formatter.py	SQL vagy direkt answer végső megfogalmazása.
src/data/router_tools.py	Router döntésének Python tool callá alakítása.
src/data/sql_tools.py	SQL lekérdezések és JSON-kompatibilis result konverzió.
src/database/config.py	Adatbázis URL előállítás.
src/database/engine.py	SQLAlchemy engine és session factory.
src/prompts/system.py	Router és formatter system promptok.
index.html	Fejlesztői tesztkliens, nem production UI.
.env.sample	Titokmentes konfigurációs minta a valós .env kulcsai alapján.